Documento de Arquitetura de Software - P4 Doctors

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
16/04/2026	1	Pablo Cardoso, Ruan Alves, Said Salles	Construção inicial do documento	1.0

Sumário

1. INTRODUÇÃO 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 4
2. REPRESENTAÇÃO ARQUITETURAL 4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS 5
4. VISÃO DE CASOS DE USO 5
4.1 Casos de Uso significantes para a arquitetura 5
5. VISÃO LÓGICA 7
5.1 Visão Geral – pacotes e camadas 7
6. VISÃO DE IMPLEMENTAÇÃO 9
6.1 Caso de Uso 9
6.1.1 Diagrama de Classes 9
6.1.2 Diagrama de Sequência 10
7. VISÃO DE IMPLANTAÇÃO 11
8. DIMENSIONAMENTO E PERFORMANCE 11
8.1 Volume 11
8.2 Performance 11
9. QUALIDADE 11

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema P4 Doctors, utilizando diversas visões de arquitetura para representar diferentes aspectos do software. O objetivo deste documento é capturar, comunicar e justificar as decisões arquiteturais significativas tomadas pela equipe de engenharia, servindo como guia fundamental para o desenvolvimento, implantação e manutenção da plataforma.

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao ecossistema do P4 Doctors, abrangendo a interface de usuário (Front-end renderizado no servidor), as regras de negócio em Python e o modelo de persistência relacional na nuvem. Ele não engloba integrações de hardware médico físico ou softwares externos de operadoras de saúde.

1.3 Definições, Acrônimos e Abreviações

- **DBaaS (Database as a Service):** Modelo de computação em nuvem que fornece acesso a um banco de dados sem a necessidade de configurar hardware físico ou infraestrutura local.
- **SSR (Server-Side Rendering):** Técnica onde a interface web é processada e renderizada no servidor antes de ser enviada ao navegador do cliente, aumentando a segurança do código fonte.
- **RBAC (Role-Based Access Control):** Controle de acesso baseado em papéis, garantindo que usuários (Médicos, Secretárias, Administradores) acessem apenas os recursos permitidos para suas funções.

1.4 Referências

[1] Metodologia de Arquitetura: KRUCHTEN, Philippe. *The "4+1" View Model of Software Architecture*. IEEE Software, vol. 12, no. 6, pp. 42-50, Novembro 1995.

[2] Interface e Aplicação (Front-End/SSR): STREAMLIT INC. *Streamlit Documentation: A faster way to build and share data apps*. Disponível em: <https://docs.streamlit.io>. Acesso em: 15 abr. 2026.

[3] Persistência e Banco de Dados em Nuvem (DBaaS): SUPABASE. *Supabase Documentation: The Open Source Firebase Alternative*. Disponível em: <https://supabase.com/docs>. Acesso em: 15 abr. 2026.

[4] Motor de Banco de Dados Relacional: POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 15 abr. 2026.

[5] Segurança e Criptografia: OWASP (Open Web Application Security Project). *OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks*. (Referência para as políticas de criptografia com Bcrypt e controle de *Session Timeout*). Disponível em: <https://owasp.org/Top10/>.

[6] Conformidade Legal: BRASIL. *Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)*. Diário Oficial da União, Brasília, DF, 15 ago. 2018. (Referência para as decisões de Exclusão Lógica e isolamento de dados clínicos).

2. REPRESENTAÇÃO ARQUITETURAL

A arquitetura do P4 Doctors baseia-se primordialmente no **Estilo Arquitetural Cliente-Servidor em Camadas (Layered Architecture)**, operando em um modelo de nuvem (*Cloud-native*).

Este documento detalhará o sistema utilizando o modelo "4+1" [KRU41], adaptado para uma aplicação moderna de renderização do lado do servidor (SSR). As visões utilizadas para representar a arquitetura são:

Visão	Público	Área	Modelo da MDS
Lógica	Arquitetos de Software, Analistas de Sistemas e Desenvolvedores.	Estrutura estática do software, separação das camadas lógicas (Apresentação, Regras de Negócio e Persistência) e organização	Diagrama de Classes Diagrama de Pacotes

		conceitual dos dados.	
Processo	Arquitetos de Software, Desenvolvedores e Engenheiros de Performance.	Comportamento do sistema em tempo de execução, controle de concorrência, comunicação síncrona/assíncrona e fluxo de segurança/autenticação.	Diagrama de Sequência Diagrama de Atividades
Implementação	Desenvolvedores, Líderes Técnicos e Engenheiros de DevOps.	Organização dos artefatos de software, módulos em Python (ex: arquivos `.py`), gerenciamento de dependências e controle de versão.	Diagrama de Componentes
Implantação	Engenheiros de Infraestrutura, Administradores de Banco de Dados (DBA) e DevOps.	Topologia de rede, nós físicos e virtuais de hospedagem na nuvem (Servidor Web Streamlit e SGBD Supabase), requisitos de hardware e comunicação externa.	Diagrama de Implantação
Caso de Uso	Clientes, Gerentes de Projeto, Testadores (QA) e Analistas de Requisitos.	Compreensão geral das funcionalidades do sistema a partir da interação dos	Diagrama de Casos de Uso

		usuários finais (Atores) para validação do escopo arquitetural.	
Dados	Administradores de Bases de Dados (DBA), Arquitetos de Dados e Programadores.	Estrutura de armazenamento persistente, modelagem relacional (tabelas, herança, chaves estrangeiras) e integridade dos dados clínicos.	Modelo Entidade-Relacionamento (DER/ERD) Dicionário de Dados

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve as escolhas tecnológicas centrais e fornece a **justificativa arquitetural** para cada decisão, evidenciando o impacto significativo dessas escolhas na performance, segurança e viabilidade do projeto.

Requisito	Solução	Justificativa
Linguagem	Python (3.10+)	A escolha do Python baseia-se na sua alta legibilidade e na vasta disponibilidade de bibliotecas orientadas a manipulação de dados (ex: Pandas). Como o sistema possui um forte componente de Dashboards financeiros para os médicos, o Python acelera a entrega de valor analítico em comparação com linguagens verbosas como Java ou C#.
Plataforma	Streamlit	Adotamos o Streamlit para abstrair a complexidade do *Front-End* tradicional (HTML/CSS/JS/React). O Streamlit opera via *Server-Side Rendering* (SSR), o que significa que o código de negócio

		nunca é exposto ao navegador do usuário. Isso reduz a superfície de ataque e acelera o ciclo de desenvolvimento da interface clínica.
Segurança	PostgreSQL (via Supabase)	O setor de saúde exige integridade relacional absoluta e trilhas de auditoria imutáveis. O PostgreSQL foi escolhido por ser o SGBD relacional open-source mais avançado, suportando nativamente tipos complexos (UUID, Timestamptz) necessários para a agenda médica. O Supabase entra como DBaaS para eliminar a sobrecarga de gerenciamento de infraestrutura da equipe.
Persistência	Bcrypt e Controle de Sessão em Memória	Devido à LGPD e à sensibilidade dos prontuários, a arquitetura restringe o armazenamento de senhas ao formato *hash*(Bcrypt com *salt*). A arquitetura não utiliza *cookies* persistentes de longa duração; o controle de estado e *timeout* é gerenciado dinamicamente na memória do servidor web para mitigar sequestro de sessão (*Session Hijacking*).

Tabela 2 – Exemplo de requisitos e restrições

4. VISÃO DE CASOS DE USO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

4.1 Casos de Uso significantes para a arquitetura

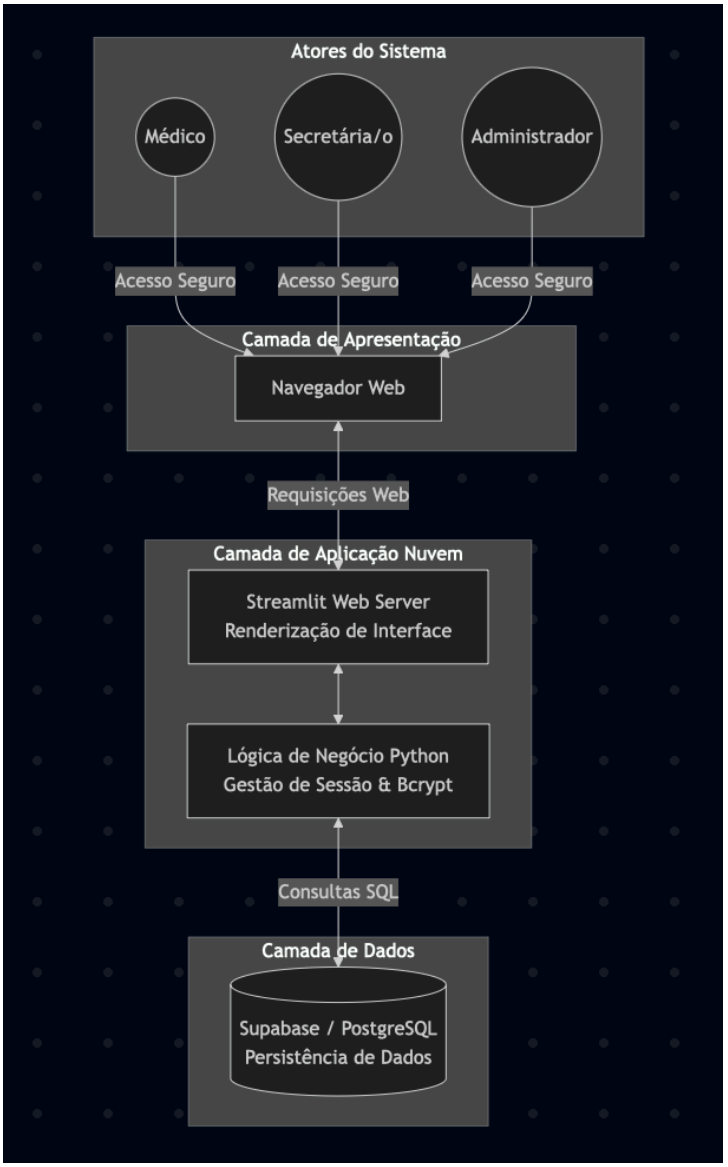

Figura 1 - Arquitetura da solução

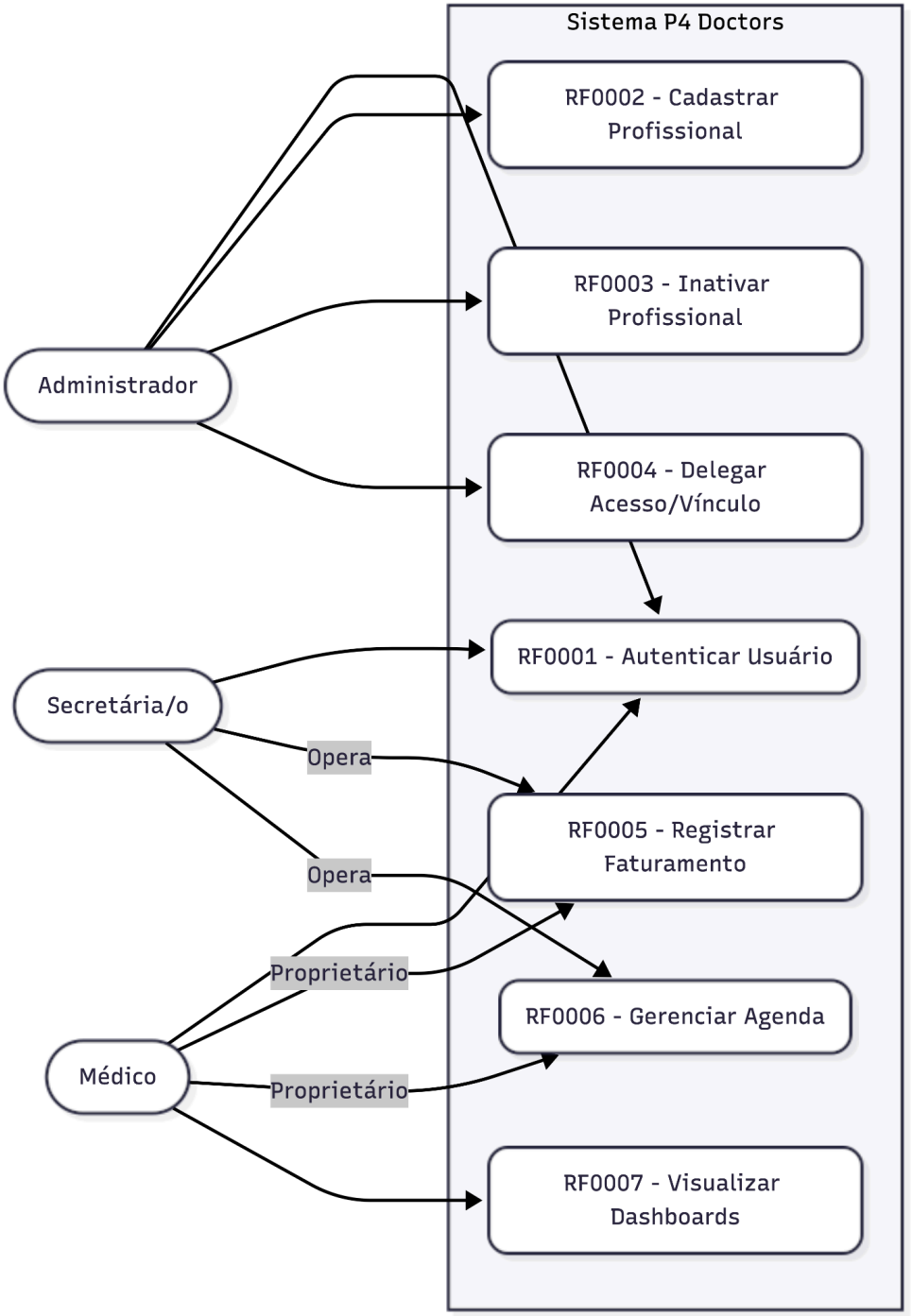

Figura 2 - Diagrama Geral de Casos de Uso evidenciando a interação de todos os atores com o escopo do sistema

5. VISÃO LÓGICA

A Visão Lógica do P4 Doctors descreve a organização conceitual do sistema, focando na decomposição funcional e na estrutura das classes. A arquitetura foi desenhada seguindo o padrão de **Arquitetura em Camadas**, promovendo o baixo acoplamento e facilitando a manutenção futura do sistema.

5.1 Visão Geral – pacotes e camadas

O sistema está dividido em três camadas principais, cada uma com responsabilidades bem definidas:

1. **Camada de Apresentação (UI):** Responsável por toda a interação com o usuário final. No P4 Doctors, esta camada é composta pelos scripts Streamlit que geram as interfaces, menus e formulários.
2. **Camada de Aplicação / Domínio:** Contém a lógica de negócio do sistema. Aqui residem as validações de regras clínicas, o processamento dos cálculos financeiros e a gestão das permissões de acesso (RBAC).
3. **Camada de Infraestrutura / Dados:** Gerencia a persistência dos dados e a comunicação externa. É responsável pela conexão com o Supabase, execução de queries SQL e gestão de credenciais.

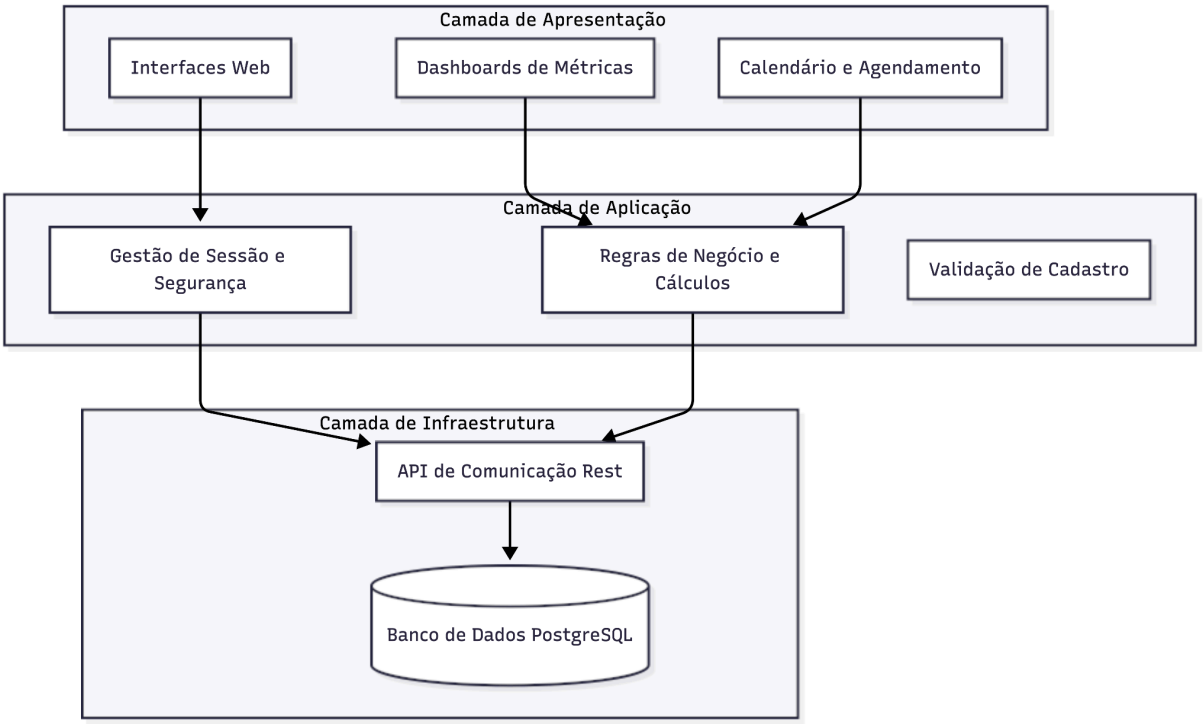

Figura 3 – Exemplo de Diagrama de Camadas da Aplicação

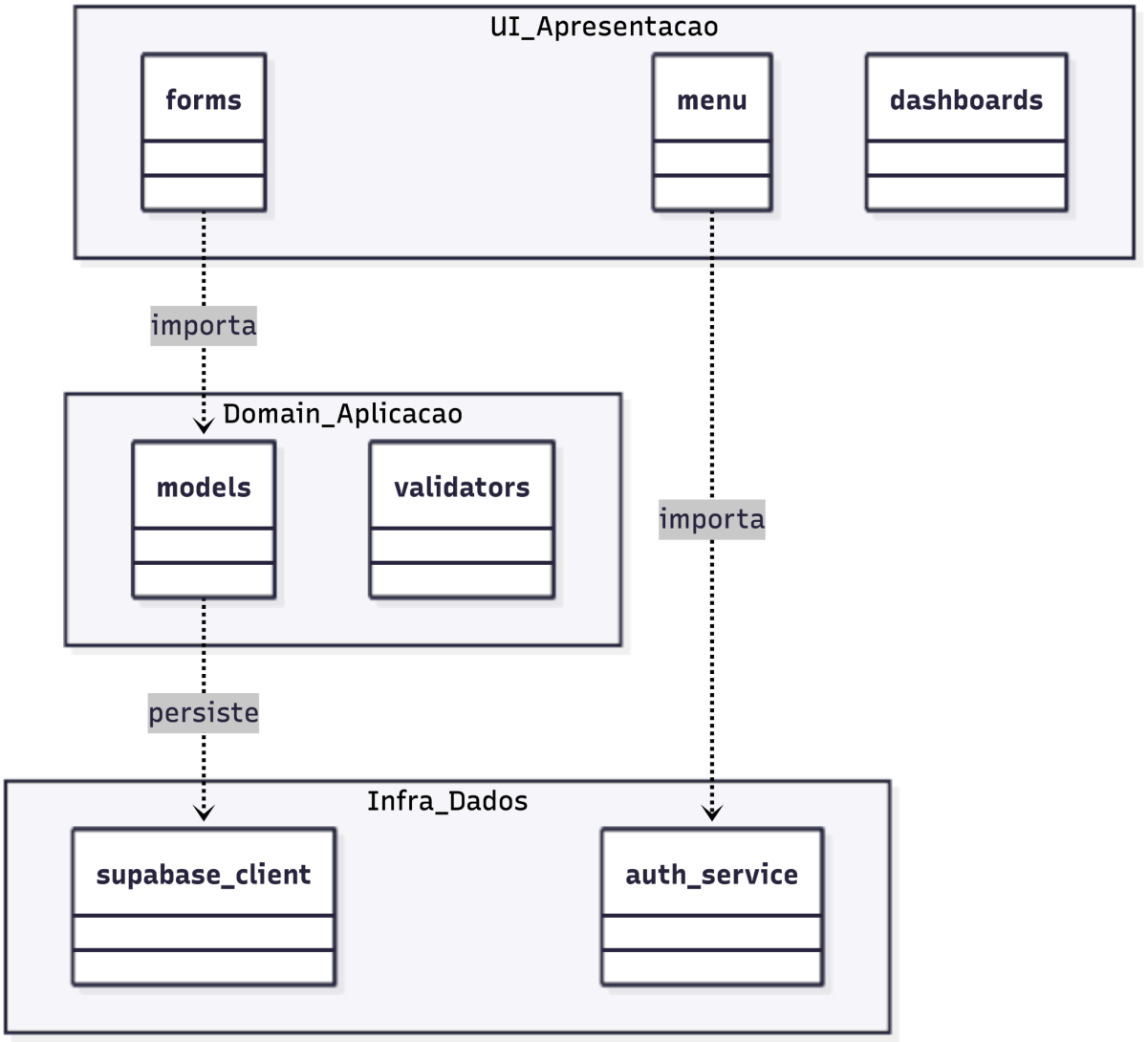

Figura 4 – Exemplo de Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

A Visão de Implementação descreve a organização dos módulos de software na camada de desenvolvimento e a dinâmica de interação entre os objetos para realizar as funcionalidades do sistema. Esta visão é ilustrada através do detalhamento do caso de uso mais crítico da plataforma.

6.1 Caso de Uso [UC01] - Resgistrar Atendimento Clínico (Faturamento)

Este caso de uso representa o núcleo operacional do P4 Doctors, onde os dados clínicos e financeiros são persistidos, garantindo a integridade referencial e a trilha de auditoria.

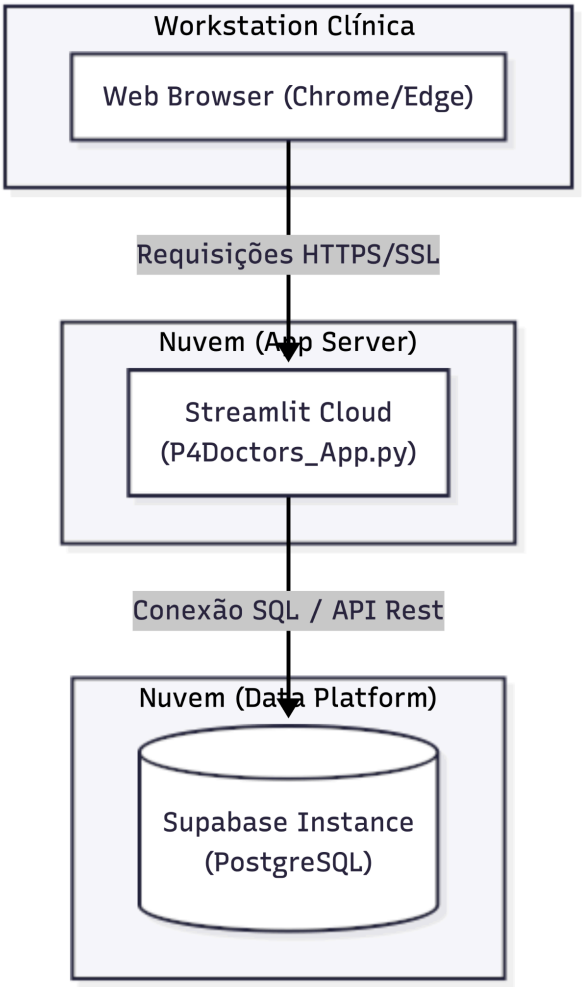

Figura 5 – Exemplo de Diagrama de Classes

6.1.1 Diagrama de Classes

O diagrama abaixo representa a estrutura estática do sistema, utilizando o padrão de **Herança de Tabela Única (Single Table Inheritance)** lógica para a entidade `Pessoa`, garantindo que médicos, pacientes e secretárias compartilhem atributos base enquanto mantêm suas especializações.

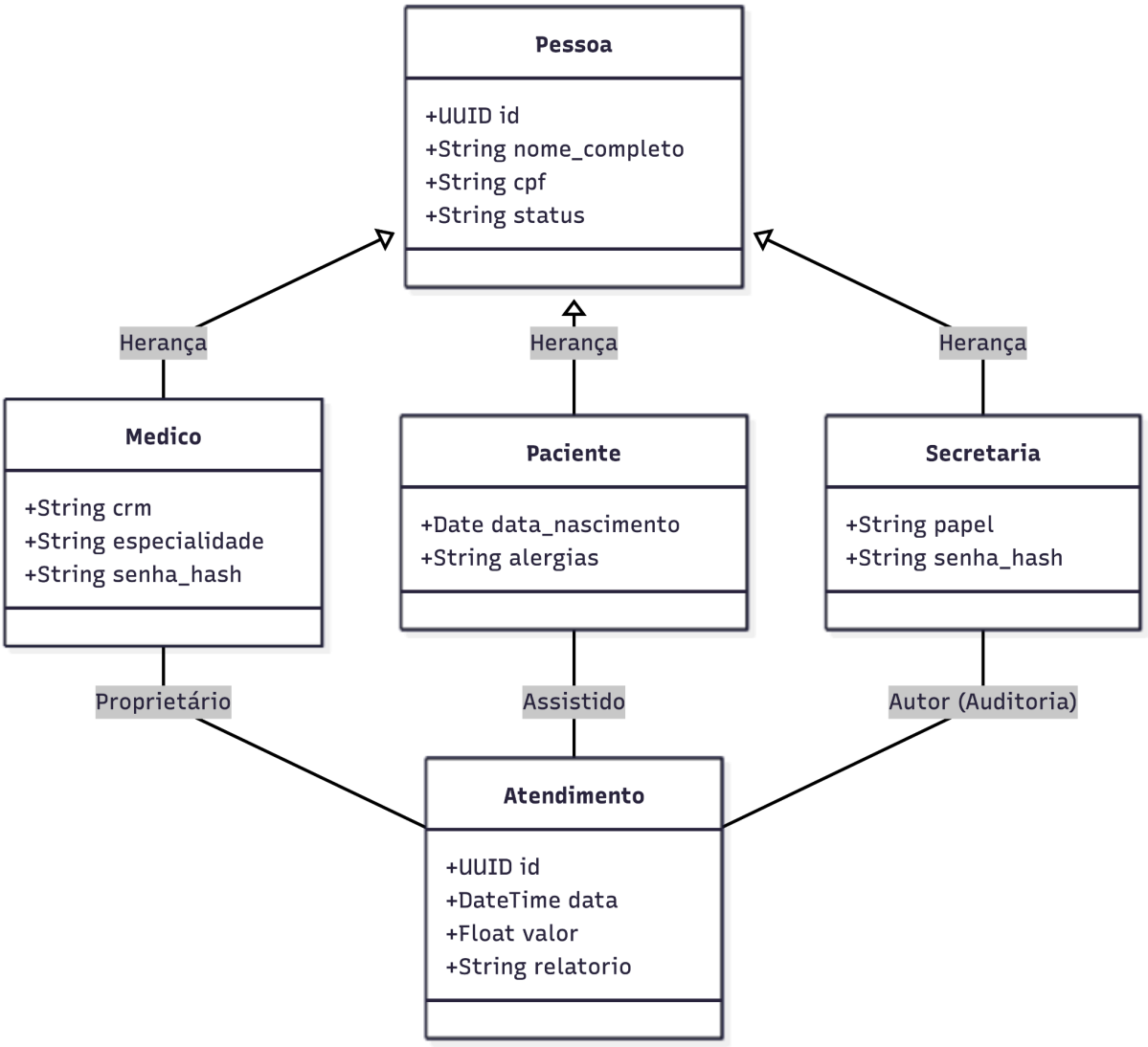

Figura 6 – Exemplo de Diagrama de Classes

6.1.2 Diagrama de Sequência

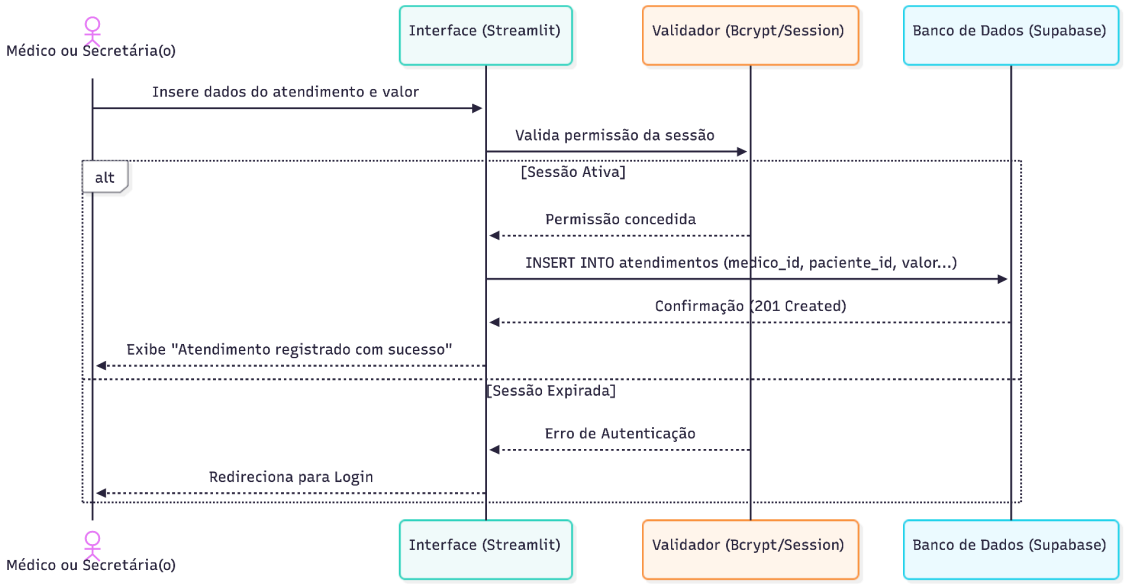

Figura 7 – Exemplo de Diagrama de Sequência

7. VISÃO DE IMPLANTAÇÃO

A Visão de Implantação descreve a topologia física dos nós de hardware, as configurações de rede e os artefactos de software que neles residem. Para o sistema P4 Doctors, a infraestrutura é baseada em serviços de nuvem (*Cloud-native*), eliminando a necessidade de servidores físicos locais nas instalações da clínica.

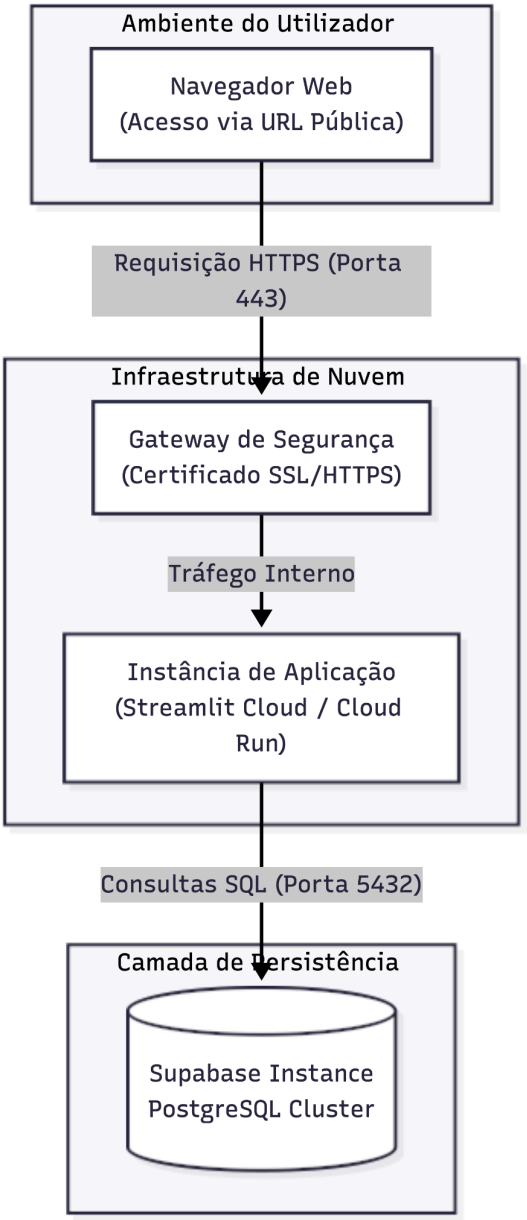

Figura 8 – Exemplo de Diagrama de Implantação

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Enumerar os itens relativos ao volume de acesso aos recursos da aplicação:

- **Número estimado de utilizadores:** 50 (profissionais clínicos, equipe de secretárias(os) e administração, considerando o perfil de uma clínica de médio porte).
- **Número estimado de acessos diários:** 800 transações (agendamentos, consultas de processos clínicos, inserções financeiras e logins).
- **Número estimado de acessos por período (mês):** 17.600 requisições (estimativa com base em 22 dias úteis de funcionamento).
- **Tempo de sessão de um utilizador:** Turno contínuo de 8 horas, sujeito a um mecanismo de *timeout* por segurança que força a reautenticação após 30 minutos de inatividade.

8.2 Performance

Enumerar os itens referentes à resposta esperada do sistema:

- **Latência P95 (95% das requisições):** Operações transacionais (CRUD) como "Registar Paciente" ou "Adicionar Faturamento" devem ser concluídas num tempo máximo de 800 ms.
- **Latência P99 (99% das requisições):** Casos extremos não devem ultrapassar 1.500 ms.
- **Processamento Analítico:** Geração de *dashboards* mensais para a administração, que exigem cálculos agregados (`SUM`, `GROUP BY`) e varrimento de tabelas grandes, possui um limite de tolerância de 3.000 ms (mitigado pelo uso de *cache* no servidor).

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	Capacidade de absorver o aumento de utilizadores e dados sem degradação do tempo de resposta.	Arquitetura *Cloud-native*. A infraestrutura em nuvem (App Server e Supabase) permite alocação dinâmica de recursos (escala vertical/horizontal) à medida que a clínica cresce.
Confiabilidade, Disponibilidade	Garantia de que o sistema se mantém acessível e operante durante os horários de consulta e plantão.	Utilização de provedores na nuvem geridos (PaaS e DBaaS) que oferecem replicação de dados e *Service Level Agreement*(SLA) de *uptime* superior a 99,9%.
Portabilidade	Capacidade do software ser operado a partir de diferentes	Abordagem 100% Web (*Server-Side Rendering* com Streamlit). Não requer instalação local, sendo compatível com

	dispositivos (PCs, tablets) e sistemas operativos.	qualquer dispositivo que possua um navegador moderno (Chrome, Safari, Edge).
Segurança	Proteção absoluta do sigilo médico e adequação às leis de proteção de dados (como a LGPD/RGPD).	Comunicação em trânsito encriptada via HTTPS (SSL/TLS), *passwords* protegidas por encriptação irreversível (Bcrypt) e isolamento de dados na base através de Políticas de Segurança ao Nível da Linha (*Row-Level Security* - RLS).